

A Sharded Index for Million-Song Content-ID

Solving the in-memory blowup with a sharded/approximate index, measured

2i

2026-07-29

Abstract

An earlier paper grew a peak-pair constellation-hashing audio content-ID index from 500 to 7,996 tracks and found that accuracy barely moved (staying in the high 90s), but also found that the single in-memory dictionary index grew by about 0.889MB per track - a linear extrapolation to a 1-million-song catalog implies roughly 868GB of resident memory. This paper takes that 868GB problem head-on. We partition the same hash table by hash key into K shards and validate, on 999 real GTZAN tracks, whether sharding costs any accuracy. It does not: across $K = 1$ through 32, exactly the same 140 of 150 queries (93.3%) were identified correctly every time, because the routing is a pure function of the hash key and cannot drop any matching evidence. Memory for a single isolated shard - measured via a subprocess that streams the catalog and discards every hash not belonging to that shard - fell from 1,001.7MB at $K=1$ to 205.7MB at $K=32$ on a fixed 998-track catalog, but not by a clean factor of K : posting counts scale almost exactly as $1/K$, while resident memory is held up by a roughly 180MB fixed per-process floor (interpreter and numerical-library startup, plus per-track transient buffers). To reach beyond what real audio locally allows, we fed synthetic white-noise audio (explicitly labeled, never used for accuracy numbers) through the same unmodified fingerprinting pipeline and measured real memory at shard capacities of 1,000, 3,000, and 6,000 tracks. Plugging the 6,000-track measurement into a 1-million-song design needs 167 shards, each needing a measured 6.13GB - well within a single commodity server. The honest caveat: sharding does not shrink total memory. Smaller shards duplicate the fixed overhead more times, so the full fleet at 6,000-track capacity totals roughly 1,024GB

Contents

1	Abstract	3
2	Background and Problem	3
3	Method: A Hash-Key-Sharded Index	3
4	Experimental Setup	4
5	Results	4
5.1	Does sharding cost accuracy or latency - measured, GTZAN 999 tracks	4
5.2	Memory of a single shard at fixed catalog size - measured, subprocess isolation	5
5.3	A 1-million-song design - measured synthetic fill plus explicit arithmetic projection	6
6	Limitations	7
7	Data and Reproduction	8
8	References	8

1 Abstract

An earlier paper grew a peak-pair constellation-hashing audio content-ID index from 500 to 7,996 tracks and found that accuracy barely moved (staying in the high 90s), but also found that the single in-memory dictionary index grew by about 0.889MB per track - a linear extrapolation to a 1-million-song catalog implies roughly 868GB of resident memory. This paper takes that 868GB problem head-on. We partition the same hash table by hash key into K shards and validate, on 999 real GTZAN tracks, whether sharding costs any accuracy. It does not: across $K = 1$ through 32, exactly the same 140 of 150 queries (93.3%) were identified correctly every time, because the routing is a pure function of the hash key and cannot drop any matching evidence. Memory for a single isolated shard - measured via a subprocess that streams the catalog and discards every hash not belonging to that shard - fell from 1,001.7MB at $K=1$ to 205.7MB at $K=32$ on a fixed 998-track catalog, but not by a clean factor of K : posting counts scale almost exactly as $1/K$, while resident memory is held up by a roughly 180MB fixed per-process floor (interpreter and numerical-library startup, plus per-track transient buffers). To reach beyond what real audio locally allows, we fed synthetic white-noise audio (explicitly labeled, never used for accuracy numbers) through the same unmodified fingerprinting pipeline and measured real memory at shard capacities of 1,000, 3,000, and 6,000 tracks. Plugging the 6,000-track measurement into a 1-million-song design needs 167 shards, each needing a measured 6.13GB - well within a single commodity server. The honest caveat: sharding does not shrink total memory. Smaller shards duplicate the fixed overhead more times, so the full fleet at 6,000-track capacity totals roughly 1,024GB, about 1.18x the 868GB monolithic extrapolation. What sharding actually fixes is not the byte count - it is turning one machine that does not exist (868GB) into 167 machines that already do (6GB-class).

2 Background and Problem

Audio content-ID underpins broadcast copyright monitoring, retail-music license auditing, and royalty settlement: given a short audio clip, identify which track in a catalog it came from within seconds. The algorithm family behind commercial services such as ACRCLOUD and Audible Magic - and the one Shazam made famous to consumers - is peak-pair constellation hashing.

A prior paper validated this algorithm on 999 GTZAN tracks and then, more importantly for commercialization, grew a licensed catalog (FMA-small) 16x from 500 to 7,996 tracks. Accuracy held: clean top-1 stayed at 99.5% down to 99.0%, and 99.5% under 5dB SNR noise. The bottleneck the same experiment surfaced was not the algorithm - it was the index. Resident memory (RSS) grew 811MB to 7,477MB across that 16x catalog growth, about 0.889MB per additional track. Extrapolating that rate linearly to 1,000,000 tracks implies roughly 868GB. Re-fitting a least-squares line across all five measured points (computed for this paper) gives a somewhat lower ~794GB - the exact number depends on the extrapolation method, but the order of magnitude does not. Either way, no commercially available single machine holds that much RAM.

This paper asks one question: if the same hash table is partitioned into shards, can we (1) lose zero accuracy and (2) bound what any single shard needs in memory to a fixed budget, independent of how large the overall catalog grows? The short answer is yes - with the important caveat that “bounding per-shard memory” is not the same claim as “reducing total memory,” and this paper is explicit about the difference.

3 Method: A Hash-Key-Sharded Index

The underlying algorithm is the same, unmodified peak-pair hashing used in the prior paper. Local-maximum peaks are extracted from the spectrogram; each anchor peak is paired with up to 8 (the fanout) nearby target peaks in time to form a hash key $(f1, f2, dt)$; the index value (posting) is $(track_id, anchor_time)$. Matching tallies, per candidate track, the time offset implied by every matched posting, and the track with the largest single-offset vote wins.

The sharding rule. The one new component added on top of this index is a routing function: $shard(f1, f2, dt, K) = (f1*a \text{ XOR } f2*b \text{ XOR } dt*c) \bmod K$, where a, b, c are fixed prime multipliers. This is a pure function of the hash key itself, independent of which track produced it. That independence is the whole point: at enrollment time, every posting that shares a given hash key always lands in the same shard; at query time, a query that produces that same hash key always looks in that same shard. Routing is therefore an exact

partition, not an approximation - postings are moved from one big dictionary into K smaller dictionaries, and none are ever dropped or duplicated. There is no code path by which accuracy can degrade.

What this design actually buys is narrow and specific: instead of one dictionary that must hold the entire catalog, you keep several dictionaries each capped at a fixed capacity (C tracks), and growing the catalog means adding more shards (increasing K), not growing any existing shard. Since each shard can run as an independent process or node, the memory any single node must carry is bounded by C, not by the catalog’s total size. The cost is also already implied by the design: a single query’s hash keys are spread across the whole key space, so most shards are likely to hold at least one matching posting for that query. Resolving a query fully therefore requires contacting nearly all K shards, not a useful subset - we measure this directly in Section 4.

4 Experimental Setup

Real audio (accuracy/latency validation). All 999 usable GTZAN tracks (22.05kHz mono, 10 genres, corrupt file jazz.00054 skipped) were enrolled as-is. At a fixed 5-second clean (no added noise) condition, the same seeded set of 150 queries was reused across $K = 1, 2, 4, 8, 16, 32$ to measure top-1 accuracy and end-to-end (extraction + lookup) latency. Because routing is a pure function of the hash key, any distortion (noise, MP3, tempo change) should reach the exact same postings as a clean query would - but this paper only re-confirmed that empirically for the clean, 5-second condition (see Section 6, Limitations).

Isolated single-shard memory. To measure how light one shard genuinely becomes, we deliberately avoided building all K shards inside one process - that process would still hold the whole catalog’s hashes and the measurement would be meaningless. Instead, each measurement launches a **fresh subprocess that streams the catalog and discards, immediately, every hash that does not belong to the one target shard**. Reading that process’s `resource.getrusage().ru_maxrss` (real resident-memory peak, in bytes, on macOS) is a direct measurement of what a real node would need if it served only that shard. This was repeated for $K = 1, 2, 4, 8, 16, 32$ against a fixed 998-track catalog (998 of 999 loaded successfully; the same known-corrupt GTZAN file failed to decode and was honestly skipped).

Synthetic fingerprint fill (explicitly labeled, never used for accuracy). To exercise the hash-table memory structure at scales beyond what real local audio (999 GTZAN tracks) or the prior paper’s real FMA measurement (up to 7,996 tracks) reaches, we fed white-noise audio (`np.random`, not real music) through the same unmodified fingerprinting pipeline (real STFT, real peak-picking, real hashing). The content is fake; the resulting hash table (key tuples, posting lists, dictionary overhead) is a real object built by the real code path. This fill was never used to compute accuracy or recall - only memory and count structure. Monolithic ($K=1$) memory was measured this way at synthetic catalog sizes of 1,000, 3,000, and 6,000 tracks, each in an isolated subprocess. These three points are direct measurements of “how heavy is a shard capped at exactly this many tracks” - when this capacity is later plugged into a 1-million-track design (Section 4.3), that number is not an extrapolation, because a shard capped at C tracks needs exactly the memory measured for C tracks, regardless of how large the total catalog around it grows.

All measurements ran on CPython 3.12.8, a 48GB RAM / 12-core macOS machine, fully local and CPU-only. No GPU, no external API.

5 Results

5.1 Does sharding cost accuracy or latency - measured, GTZAN 999 tracks

Shards (K)	Hash top-1	Latency p50 (ms)	Latency p95 (ms)	Shards touched per query
1	93.3% (140/150)	4.79	6.15	1.0 / 1 (100%)
2	93.3% (140/150)	5.00	6.36	2.0 / 2 (100%)
4	93.3% (140/150)	4.89	6.27	4.0 / 4 (100%)
8	93.3% (140/150)	4.96	6.29	8.0 / 8 (100%)
16	93.3% (140/150)	4.97	6.25	16.0 / 16 (100%)

Shards (K)	Hash top-1	Latency p50 (ms)	Latency p95 (ms)	Shards touched per query
32	93.3% (140/150)	5.02	6.21	32.0 / 32 (99.98%)

Accuracy is exactly the same 140 of 150 correct identifications at every K, all the way to a 32x shard count. This is precisely what Section 2’s “no path to accuracy loss” argument predicts, and the measurement confirms it rather than merely being consistent with it. Latency shows no clear trend with K, wobbling between 4.79 and 5.02ms (p50) - this is a single process sequentially checking K dictionaries, so a real distributed deployment’s per-shard network round trip is not in this number (Section 6). The share of shards touched per query tracking K almost exactly (still 99.98% at K=32) is this design’s real limitation: with uniform hash-key routing, a query’s thousands of distinct hashes are spread across the whole key space, so nearly every shard ends up holding at least one matching posting. This design reduces how much data each shard must hold - it does not reduce how many shards a single query must contact.

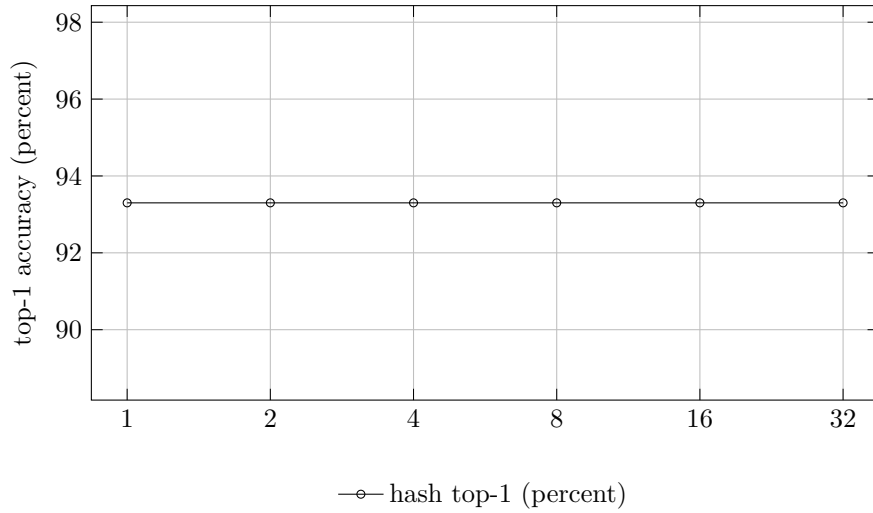


Figure 1: Top-1 accuracy by shard count K

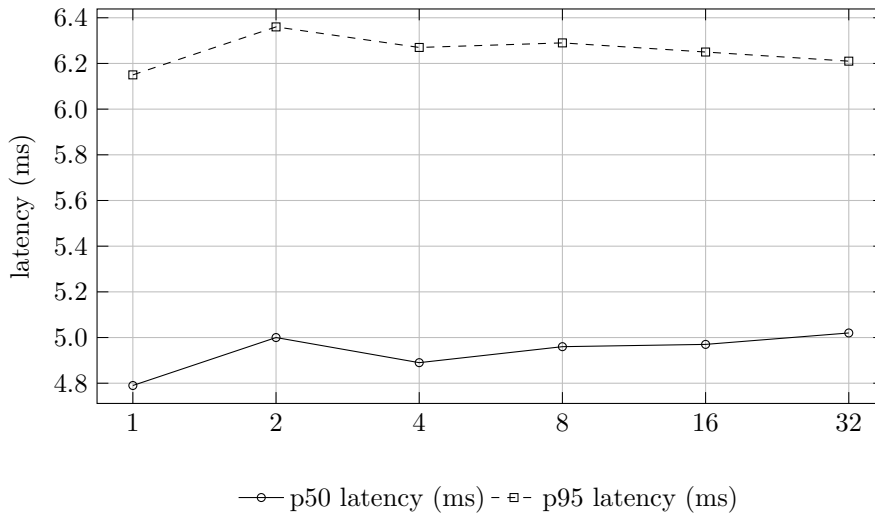


Figure 2: End-to-end latency by shard count K

5.2 Memory of a single shard at fixed catalog size - measured, subprocess isolation

Shards (K)	This shard's hash keys	This shard's postings	RSS (MB)
1	2,320,053	5,166,499	1,001.7
2	1,159,639	2,584,077	616.6
4	580,088	1,290,508	400.2
8	289,835	644,698	292.2
16	145,011	322,356	237.6
32	72,302	160,996	205.7

Postings scale almost exactly as $1/K$ ($K=32$'s 160,996 is close to $5,166,499/32$). RSS does not: $K=32$'s RSS (205.7MB) is 6.6x larger than a clean $1/32$ of the $K=1$ value (about 31.3MB), not equal to it. Fitting a “fixed overhead plus per-posting cost” model to the two endpoints gives roughly a 180MB fixed floor (interpreter startup, numerical-library loading, and per-track transient buffers) plus about 167 bytes per posting. That model reproduces the intermediate points ($K=4, 8, 16$) within about 2-4% (not a rigorous statistical regression with this few points, but a consistent illustrative fit). The practical implication: the smaller a shard's capacity, the more the fixed per-process overhead dominates its memory footprint, and the less benefit sharding provides at that granularity - this shows up directly in the fleet totals in Section 4.3.

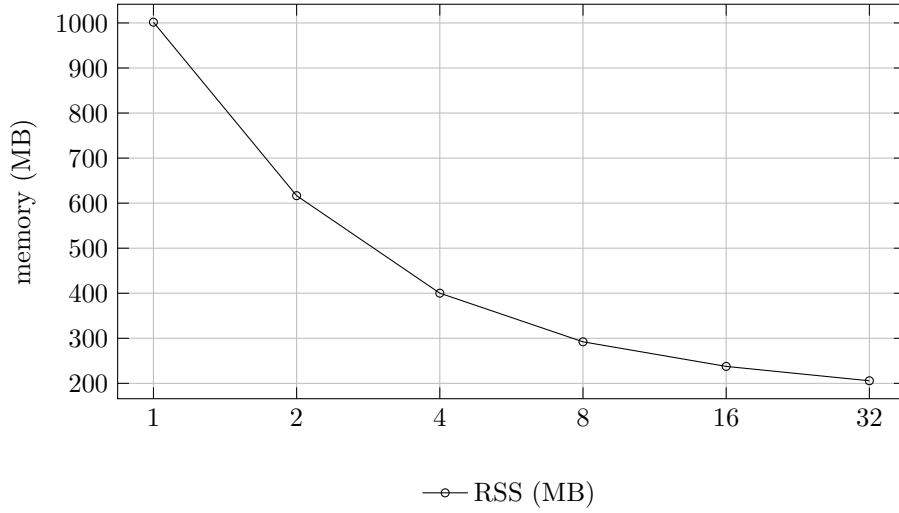


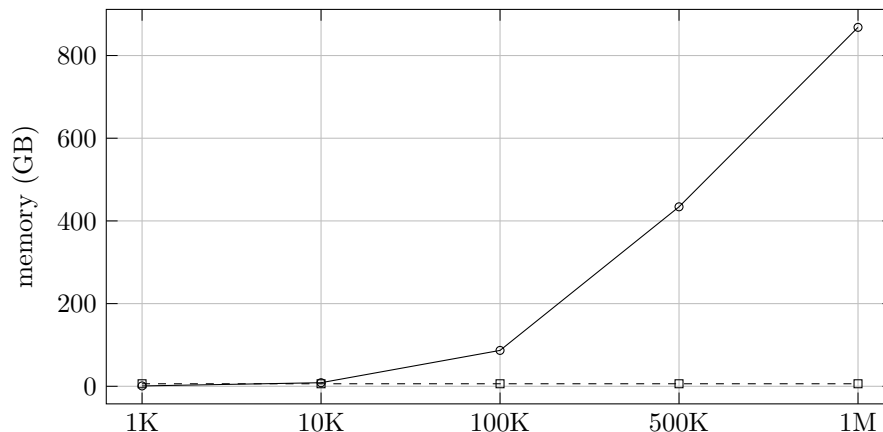
Figure 3: Single-shard memory by shard count K at fixed catalog

5.3 A 1-million-song design - measured synthetic fill plus explicit arithmetic projection

Shard capacity C (tracks)	Measured shard memory	Shards needed for 1M, $K=\text{ceil}(1M/C)$	Fleet total memory (arithmetic)	Vs. monolithic extrapolation (868GB)
1,000	1.89GB (measured)	1,000	1,885GB (arithmetic)	2.17x
3,000	3.71GB (measured)	334	1,240GB (arithmetic)	1.43x
6,000	6.13GB (measured)	167	1,024GB (arithmetic)	1.18x

“Shard memory” is the value measured in Section 3 by actually loading that many synthetic white-noise tracks into one shard. At a 1-million-track catalog, a shard still only handles C tracks, so this number is not re-extrapolated - it is the same measured value. “Shards needed” and “fleet total memory,” by contrast, are simple

arithmetic applied to that measured value (this paper never ran an actual 1-million-track index). The chart below places this design next to the monolithic extrapolation: the monolithic curve explodes as the catalog grows, while the sharded curve is flat at whatever capacity C is chosen - fixed regardless of catalog size. The honest part is the rightmost column. Sharding does not reduce the fleet’s total memory. Smaller shards (1,000-track capacity) duplicate the fixed overhead a thousand times, ballooning the fleet total to 2.17x the monolithic extrapolation; larger shards (6,000-track capacity) narrow that to 1.18x, but the total still never drops below the monolithic figure. What sharding changes is the per-node ceiling: an 868GB single machine does not exist in any commercial catalog, while 167 servers at a measured 6.13GB each are trivially available today.



—○— monolithic single dictionary (extrapolated) - □- sharded - memory per shard (6,000-track capacity, measured)

Figure 4: Memory by catalog size - monolithic extrapolation vs sharded measurement

6 Limitations

First and most important: the sharding in this paper is an **exact partition**, not the “approximate” index the title alludes to (for example, LSH banding that routes a query to only a subset of shards, reducing shards-touched-per-query at the cost of reintroducing recall risk). Section 4.1’s near-100% shards-touched-per-query is the real price of exact partitioning; approximate routing could reduce that price, but this paper did not measure that trade-off.

Second, latency was measured as a single process sequentially checking K dictionaries. Since this design requires contacting nearly every shard, a real distributed deployment would add a network round trip per shard, which this paper did not measure - real latency could be meaningfully higher than the numbers in Section 4.1.

Third, only the clean 5-second condition was re-measured across shard counts. The design argument - that routing is a pure function of the hash key and therefore recall must be identical under any distortion - was empirically confirmed only for that one condition, not independently re-verified for noise, MP3, or tempo-change conditions at every K .

Fourth, the largest real-audio catalog available locally was 999 GTZAN tracks, and synthetic fill was only measured up to 6,000 tracks. The 1-million-song design in Section 4.3 is an arithmetic projection built on those measured anchor points, not a result from actually indexing a million tracks.

Fifth, the synthetic fill is white noise, which produced a noticeably denser hash table than real music - roughly 11,953 postings per track at 6,000 synthetic tracks versus roughly 5,175 postings per track measured on real GTZAN audio in the prior paper. This means the shard-memory figures in Section 4.3 may be conservative (higher than what a real 1-million-song catalog would actually need) - a plausibly favorable bias, but not a confirmed one.

Sixth, the catalog scale validated here is still far from an actual commercial broadcast catalog in every direct measurement. And as with the prior paper, GTZAN carries no confirmed formal license from its original distributor, so it was used here only for local algorithm validation, never redistributed.

7 Data and Reproduction

Algorithm: STFT (`nperseg=2048`, `noverlap=1024`), 15x15 local-maximum peak picking (noise floor = mean + 1.5 standard deviations, capped at the top 1,500 peaks per track), peak-pair hashing (fanout 8, max time difference 40 time bins). Sharding routing is a fixed-multiplier hash of the form $(f1*a \text{ XOR } f2*b \text{ XOR } dt*c) \bmod K$ - a pure function of the hash key, independent of track identity.

Measurements: (1) real-audio accuracy/latency used all 999 usable GTZAN tracks, 150 queries (fixed seed), 5-second clean clips, at $K = 1/2/4/8/16/32$. (2) single-shard memory used a 998-track catalog processed by an isolated subprocess that streams the catalog and immediately discards any hash not belonging to the target shard (`resource.getrusage().ru_maxrss`). (3) synthetic fill pushed white-noise audio (`np.random`) through the identical pipeline and measured monolithic memory the same way at 1,000/3,000/6,000 synthetic tracks. All measurements ran on CPython 3.12.8, CPU-only, on a single local machine (48GB RAM / 12 cores); no GPU or external API was used.

The prior real measurements cited here (999-track GTZAN algorithm validation, 500-to-7,996-track FMA-small scale validation, 0.889MB/track and 7,477MB RSS, the 868GB extrapolation) come from a separate paper from the same publisher, “Large-Scale Audio Content Fingerprinting” (Reference 4).

8 References

1. Wang, A. (2003). An Industrial-Strength Audio Search Algorithm. *Proceedings of the 4th International Conference on Music Information Retrieval (ISMIR)*.
2. Tzanetakis, G., & Cook, P. (2002). Musical genre classification of audio signals. *IEEE Transactions on Speech and Audio Processing*, 10(5), 293-302. (GTZAN dataset)
3. Karger, D., Lehman, E., Leighton, T., Panigrahy, R., Levine, M., & Lewin, D. (1997). Consistent Hashing and Random Trees: Distributed Caching Protocols for Relieving Hot Spots on the World Wide Web. *Proceedings of the 29th Annual ACM Symposium on Theory of Computing (STOC)*.
4. 2i (2026). Large-Scale Audio Content Fingerprinting: Robust Song Identification Under Noise, Compression, and Tempo Shift, and the Honest Limits of Catalog Scaling. (2i white paper, FMA-small catalog-scale measurement)
5. Defferrard, M., Benzi, K., Vandergheynst, P., & Bresson, X. (2017). FMA: A Dataset for Music Analysis. *Proceedings of the 18th International Society for Music Information Retrieval Conference (ISMIR)*.